

Tema 6: De LLM a Agentes, Harness y tools

EBIS Business Techschool

Contenidos

- ¿Qué puede hacer un LLM?
- Plataformas de IA: La Era Agéntica
- El arnés de un agente
- ¿Qué es una herramienta?



¿Qué puede hacer un LLM?



EBIS Business Techschool

¿Qué puede hacer un LLM?

Un LLM es, en esencia, **un gigantesco predictor** del siguiente token:

Una arquitectura diseñada para analizar el texto y calcular qué debería venir después.



Al hacerlo a una escala enorme, sus predicciones se vuelven tan sofisticadas que el resultado puede parecer **conocimiento sobre cualquier tema**.

¿Qué puede hacer un LLM?



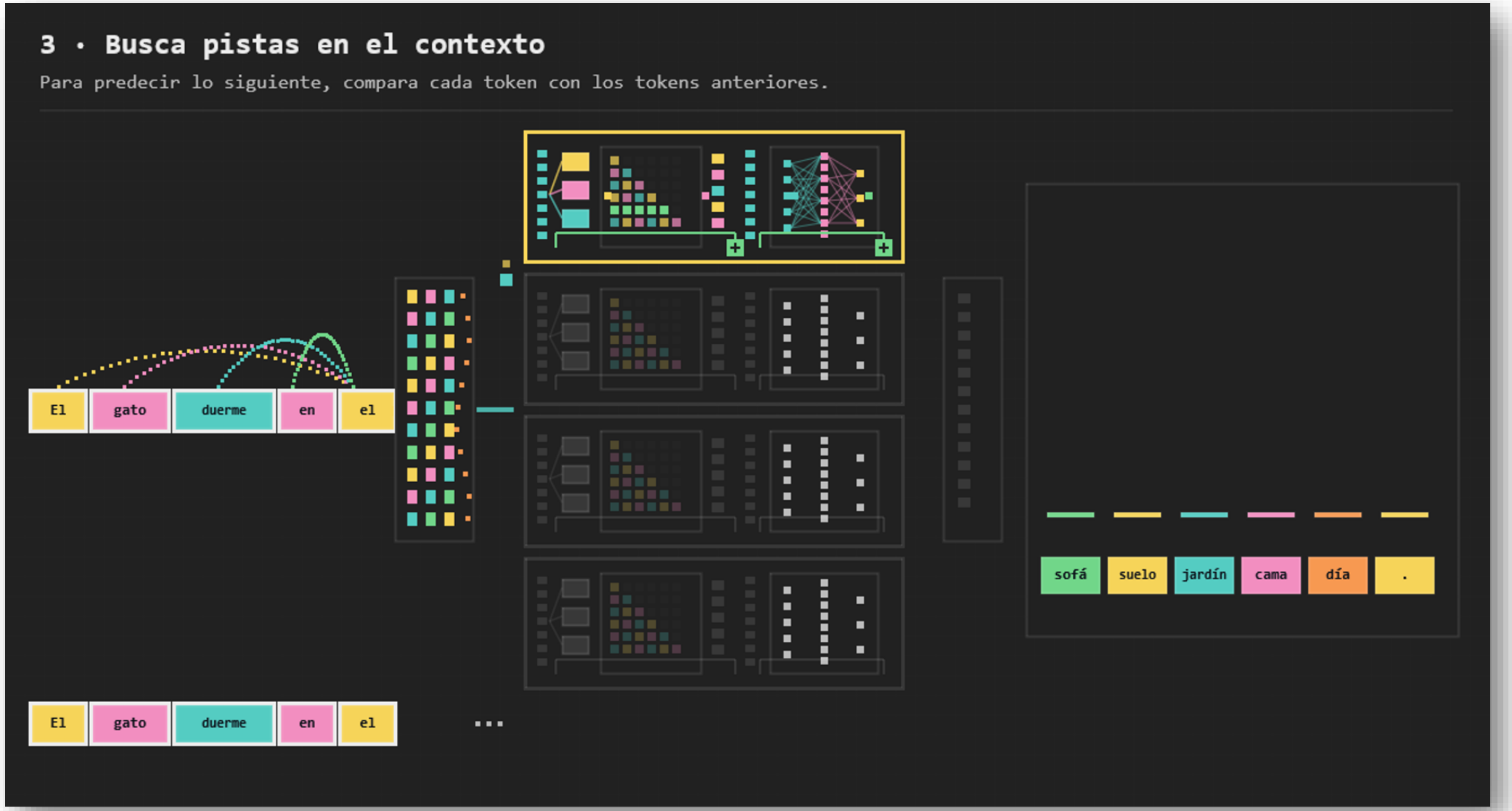
¿Qué puede hacer un LLM?

EBIS Business Techschool



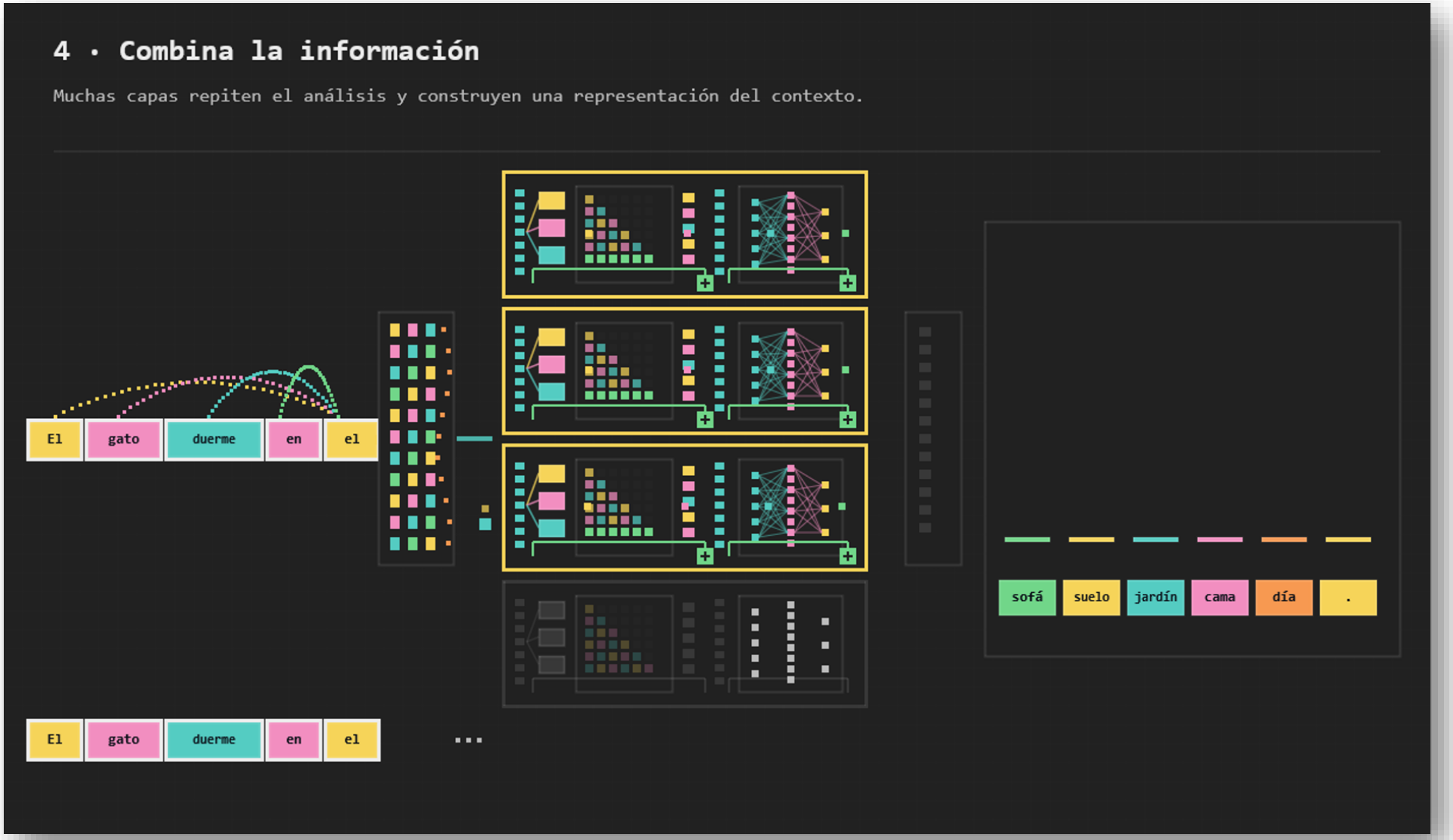
¿Qué puede hacer un LLM?

EBIS Business Techschool



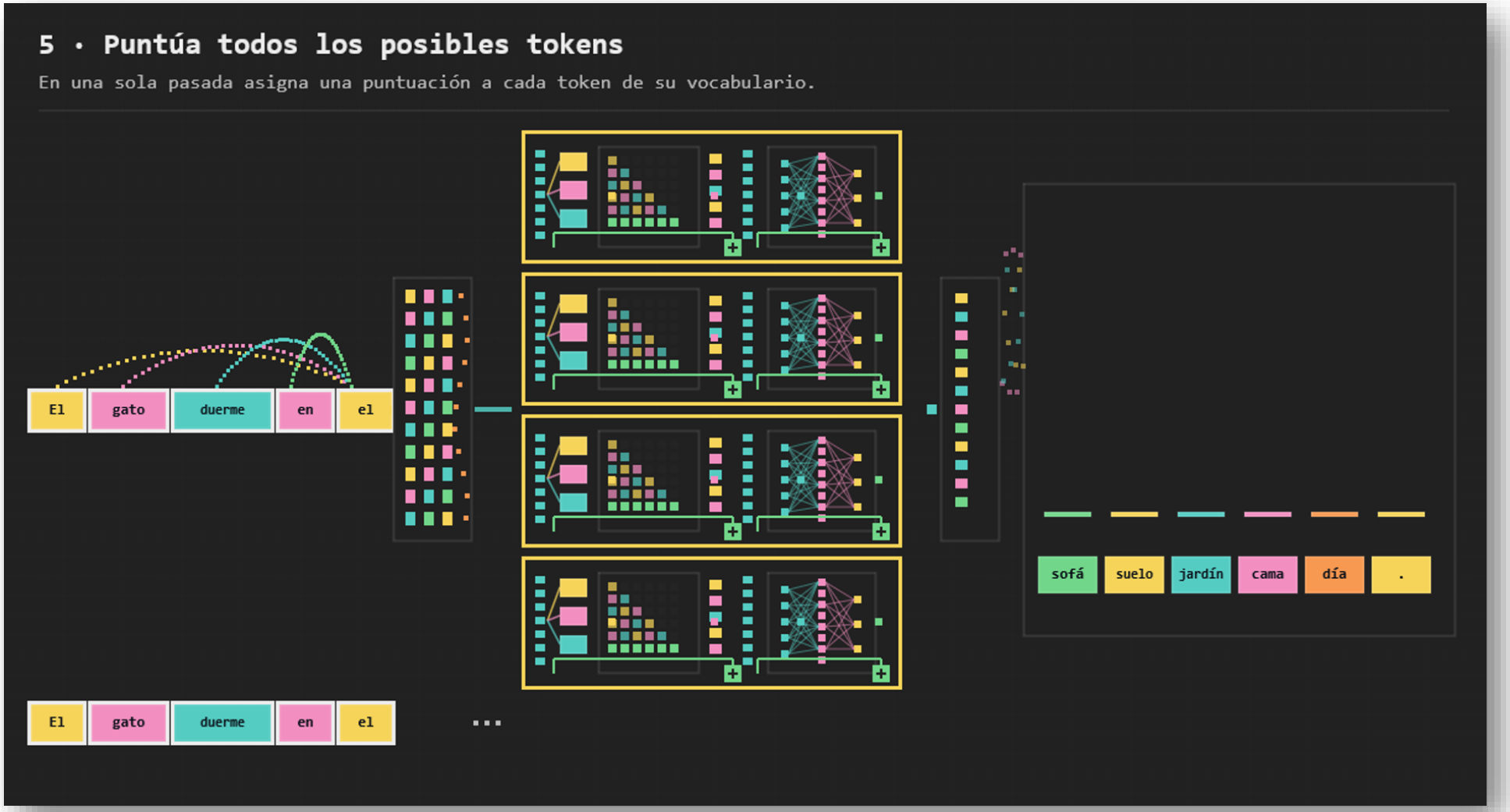
¿Qué puede hacer un LLM?

EBIS Business Techschool

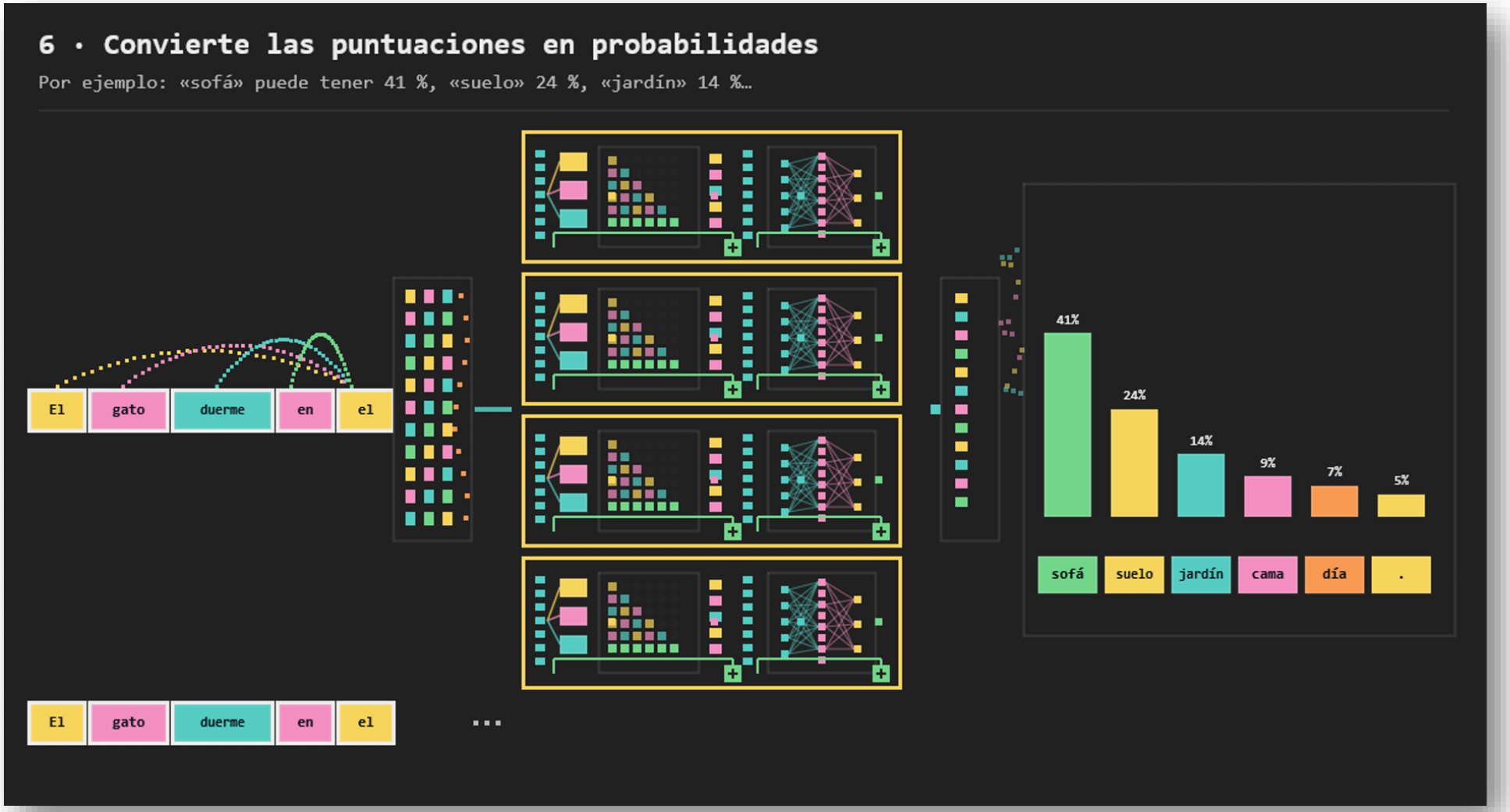


¿Qué puede hacer un LLM?

EBIS Business Techschool

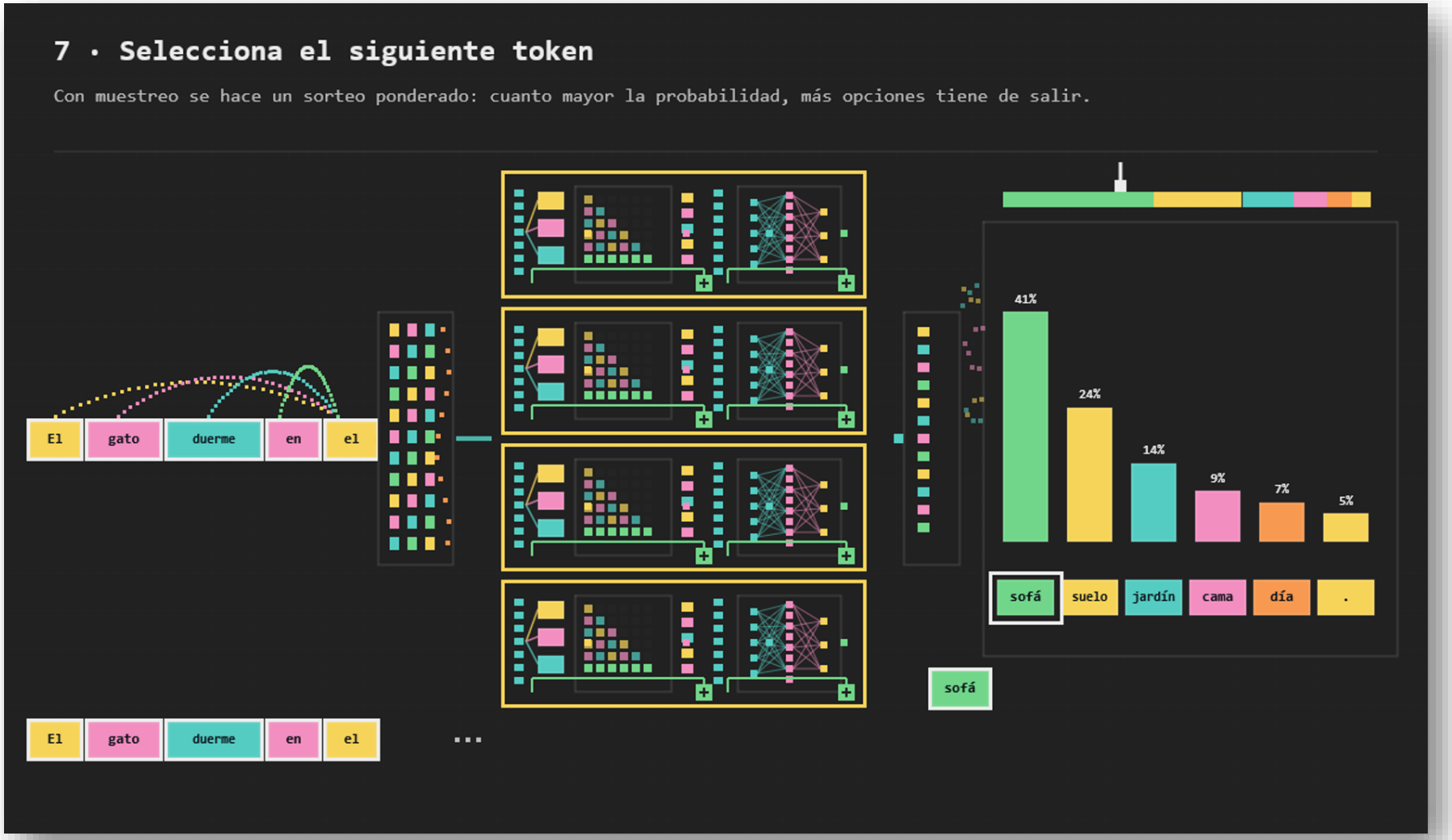


¿Qué puede hacer un LLM?



¿Qué puede hacer un LLM?

EBIS Business Techschool



¿Qué puede hacer un LLM?

Los modelos LLM **no escriben una respuesta completa** y después la muestran.

Van construyéndola token a token, calculando en cada paso **qué token es más probables dado todo lo que ha aparecido antes**.



¿Qué puede hacer un LLM?

Un LLM no busca la verdad: busca una continuación basada en probabilidades

probable no significa necesariamente **verdadero**.

Si el modelo no dispone de suficiente información, puede generar una respuesta que:

- Encaja perfectamente con el lenguaje,
- Parece razonable,
- Pero es incorrecta o inventada.



¿Qué puede hacer un LLM?

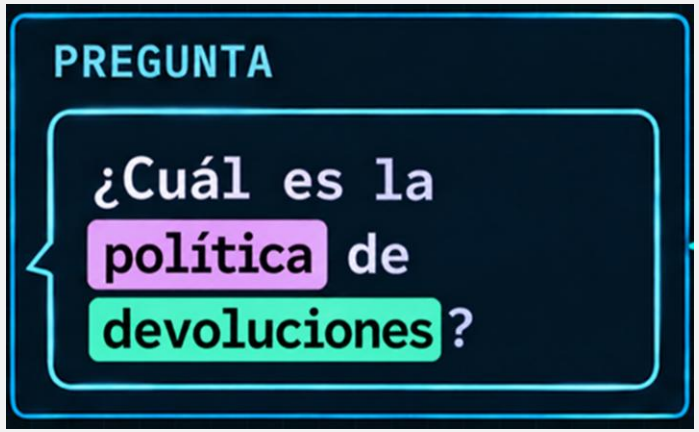
Un LLM contiene una enorme cantidad de conocimiento y patrones aprendidos.

Pero para aprovecharlos correctamente necesitamos **reducir la incertidumbre** sobre lo que queremos que haga.

La **herramienta** fundamental para hacerlo es → **CONTEXTO**

Cuanto mejor sea el contexto, mejor se orientan las probabilidades hacia una respuesta útil.

- Información relevante,
- Instrucciones,
- Restricciones,
- Resultados de herramientas.

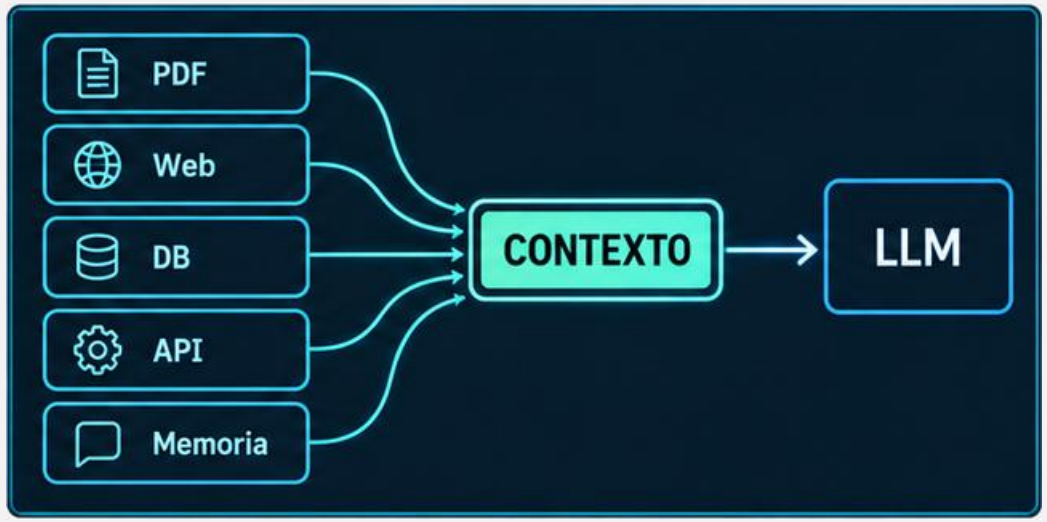


¿Qué puede hacer un LLM?

EBIS Business Techschool

Herramientas habituales

- RAGs
- Documentos y archivos
- Búsqueda Web
- Consultas a bases de datos
- APIs (Google Calendar, Slack…)
- Historial



La idea no es actualizar el modelo continuamente, sino mejorar el contexto desde el que responde

¿Algo más?

¿Qué puede hacer un LLM?

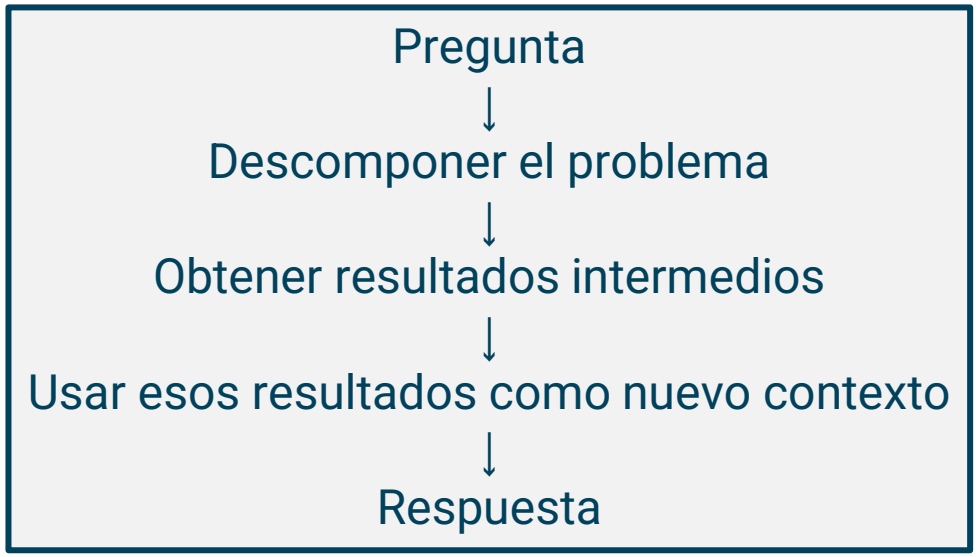
EBIS Business Techschool

¿Y si el propio modelo genera contexto?

Razonamiento (Cadenas de Pensamiento)

Hasta ahora hemos proporcionado el contexto desde fuera. Pero existe otra posibilidad, antes de responder, el modelo puede realizar **pasos intermedios que le ayuden a resolver el problema**.

En lugar de intentar predecir directamente Pregunta → Respuesta



¿Por qué el razonamiento ayuda tanto?

EBIS Business Techschool

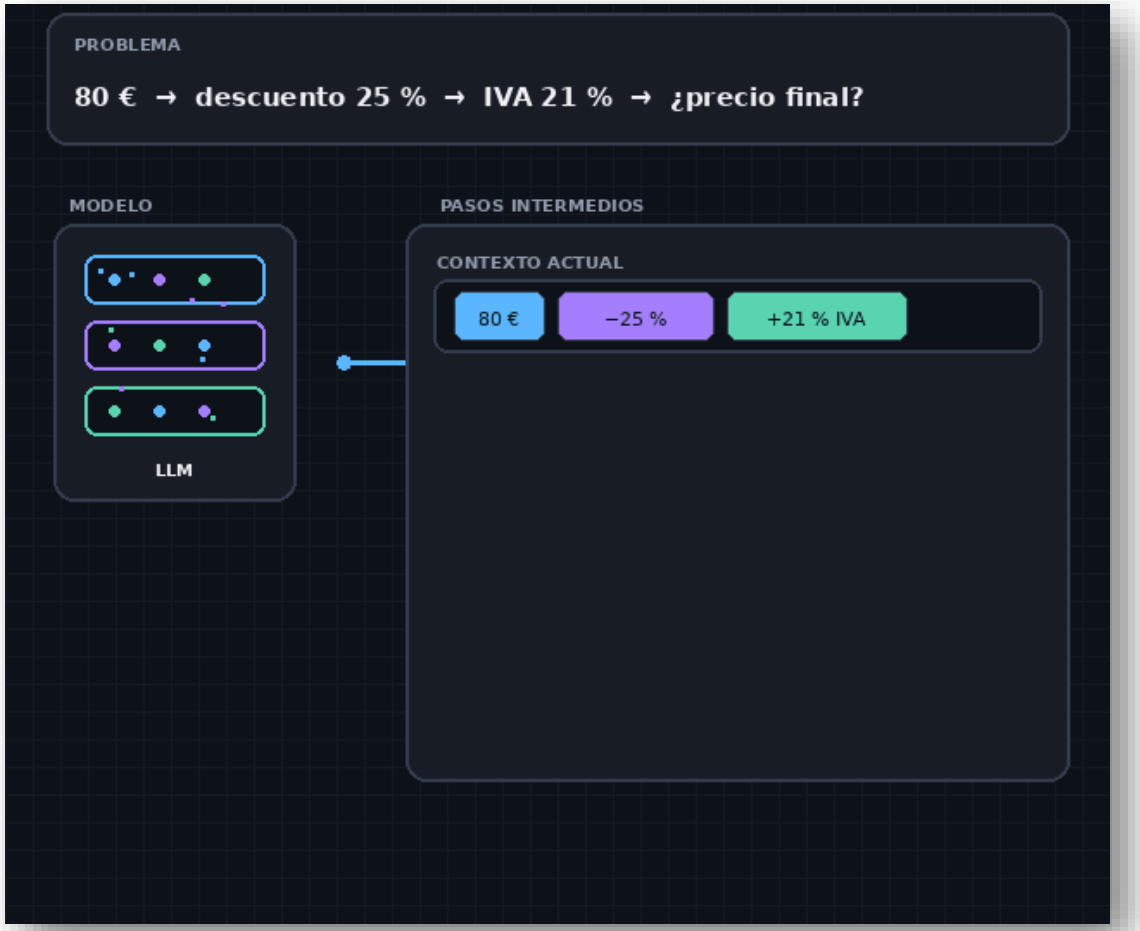
Convierte una predicción difícil en varias predicciones más sencillas

Responder directamente a un problema complejo **requiere acertar muchas relaciones a la vez**.

El razonamiento permite:

1. Dividir el problema
2. Resolver una parte
3. Utilizar ese resultado
4. Resolver la siguiente
5. Llegar finalmente a la respuesta.

Consiguiendo grandes mejoras en **matemáticas**, **lógica**, **programación**, **planificación**...



Plataformas de IA: La Era Agéntica



EBIS Business Techschool

Plataformas de IA: La Era Agéntica

Un LLM genera una respuesta a partir del contexto disponible.

Puede redactar, resumir o proponer un plan, **pero necesita capacidades externas para consultar datos o ejecutar acciones.**

Petición	Qué falta fuera del modelo
«¿Qué facturas siguen pendientes?»	Consultar el estado actual en el sistema de facturación.
«Guarda el resumen en un archivo»	Ejecutar código con acceso y permisos de escritura.

El Chat Tradicional (2022-2023)

EBIS Business Techschool

PATRÓN REACTIVO

El sistema es únicamente un motor de transformación de texto:

Espera un input, procesa y libera el output.

Un flujo de trabajo **human-in-the-loop** total:

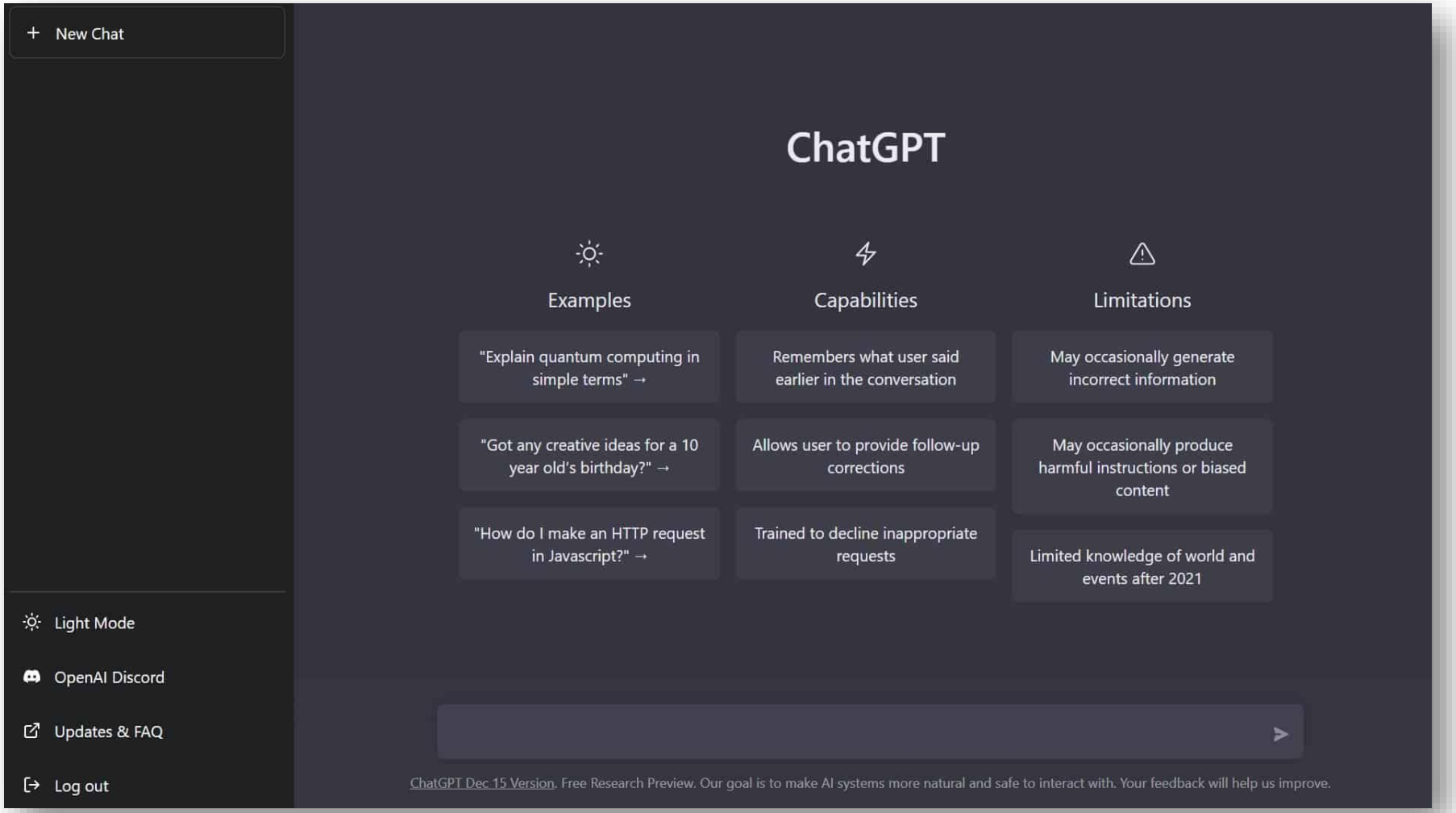
```
def chat(messages, chat_history):
    # Invocación única por turno
    response = llm.invoke(messages, chat_history)
    return response
```

LIMITACIONES

- **Conocimiento Estático (Knowledge Cutoff)**
- **Alucinaciones por falta de Grounding**
- **Copy-paste-in-the-loop**
- **Saturación y Degradación del Contexto**
- **Stateless**

El Chat Tradicional (2022-2023)

EBIS Business Techschool



Chat Tradicional: el trabajo entre turnos

Objetivo: «Revisar las facturas vencidas y guarda un resumen».

Quién	Qué hace
Persona	Exporta las facturas y las pega en la conversación.
LLM	Analiza los datos recibidos y redacta el resumen.
Persona	Verifica el contenido y lo guarda en un archivo.

El modelo ayuda con el análisis. La persona conecta cada paso con el sistema real.

El Chat Proactivo (2024)

EBIS Business Techschool

PATRÓN PROACTIVO

El sistema añade un **proceso previo de contextualización automático** (buscar en internet, leer documentos…)

```python
def chat(user_input, chat_history):
    # 1. El LLM actúa como generador de intenciones/queries
    search_queries = llm.generate_queries(user_input)

    # 2. Recuperación síncrona de datos externos (Grounding)
    retrieved_docs = vector_db.search(search_queries)

    # 3. Inyección en la ventana de contexto y respuesta final
    enriched_prompt = setup_context(user_input, retrieved_docs)
    response = llm.invoke(enriched_prompt, chat_history)

    return response
```

LIMITACIONES

- ~~*Conocimiento Estático (Knowledge Cutoff)*~~
- ~~*Alucinaciones por falta de Grounding*~~
- **Copy-paste-in-the-loop**
- **Saturación y Degradación del Contexto**
- **Stateless**
- **Ausencia de Razonamiento Iterativo**
- **Read-Only Limitation**

El Chat Proactivo (2024)

EBIS Business Techschool

what are some great ways to fix up a backyard

54 Backyard Ideas to Upgrade Your Outdoor Space
thespruce.com › backyard ideas



Transforming your backyard into a welcoming and functional space can be both enjoyable and rewarding. Here are several ideas to enhance your outdoor area:

1. **Create a Cozy Seating Area:** Arrange comfortable outdoor furniture, such as sofas, chairs, or hammocks, to establish a relaxing spot for reading or entertaining guests. Adding colorful cushions and throw pillows can infuse personality and comfort. THE SPRUCE
2. **Incorporate Outdoor Lighting:** Enhance ambiance and extend usability into the evenings by installing string lights, lanterns, or solar-powered fixtures. Soft, warm lighting can create a magical atmosphere. THE SPRUCE
3. **Add a Fire Pit:** Introducing a fire pit provides a focal point for gatherings and allows for enjoyable

Citations

The Spruce
43 Backyard Ideas on a Budget — The Spruce
August 25, 2024 — Set up a hammock and add pillows, string lighting in trees, blankets, and even a plate (or b...

The Spruce
7 Ways to Refresh Your Backyard This Summer Without Spending a Dime — The Spruce
May 18, 2024 — Power wash the patio. Clean outdoor rugs. Wipe down furniture. Wash cushion covers and...

Family Handyman
50 Brilliant Ways to Spruce Up Your Backyard This Summer
October 6, 2024 — Make Your Own Cornhole Game. Cornhole boards are a classic backyard game that is e...

Better Homes & Gardens
50 Stunning Backyard Ideas That Fit Every Kind of Space...
May 7, 2024 — Whether you're looking for patio suggestions, outdoor dining, or a fun space to relax wi...

Extra Space Storage
Your DIY Guide to a Backyard Makeover on a Budget
September 24, 2024 — Give your outdoor living space a facelift with a calming meditation space. Find a seclud...

Chat Proactivo: contexto automático

Objetivo: «Revisar las facturas vencidas y guarda un resumen».

El chat tiene acceso a una base de datos o RAG con las facturas.

Paso previsto	Resultado
1. Recuperar datos	Consulta las facturas mediante una integración.
2. Aportar contexto	Incluye los registros recuperados en la petición.
3. Generar respuesta	El LLM redacta el resumen con esos registros.

Si faltan datos, hace falta un mecanismo adicional para decidir qué consultar después.

La era de los Agentes (2025-)

EBIS Business Techschool

PATRÓN AGÉNTICO

El sistema posee un bucle de control (**ReAct**) con capacidad de **razonar**, **evaluar** errores y **alterar el estado** del mundo de forma autónoma.

Un flujo de trabajo Agent-in-the-loop:

```
def agent_chat(user_goal, chat_history, tools):
    global task_completed
    global current_context

    current_context = {
        "chat_history": chat_history,
    }

    while not task_completed:
        thought = llm.think(user_goal, current_context)

        action = llm.decide_tool(thought, tools)
        observation = execute(action)

        current_context.update(observation)

        if check_success(observation, user_goal):
            break

    return llm.generate_final_response(current_context)
```

LIMITACIONES

- ~~*Conocimiento Estático (Knowledge Cutoff)*~~
- ~~*Alucinaciones por falta de Grounding*~~
- ~~**Copy-paste-in-the-loop**~~
- ~~**Stateless**~~
- ~~**Ausencia de Razonamiento Iterativo**~~
- ~~**Read-Only Limitation**~~
- **Saturación y Degradación del Contexto**
- **Coste y Latencia (Inference Tax)**
- **Fragilidad del Bucle**

EBIS Business Techschool

PATRÓN ReAct ∞

```python
def agent_chat(user_goal, chat_history, tools):
    global task_completed
    global current_context

    current_context = {
        "chat_history": chat_history,
    }

    while not task_completed:
        thought = llm.think(user_goal, current_context)

        action = llm.decide_tool(thought, tools)
        observation = execute(action)

        current_context.update(observation)

        if check_success(observation, user_goal):
            break

    return llm.generate_final_response(current_context)
```

Fusionar la capacidad de generar cadenas de pensamiento (**Reasoning**) + la capacidad de interactuar (**Acting**) con entornos externos. El agente itera a través de tres fases

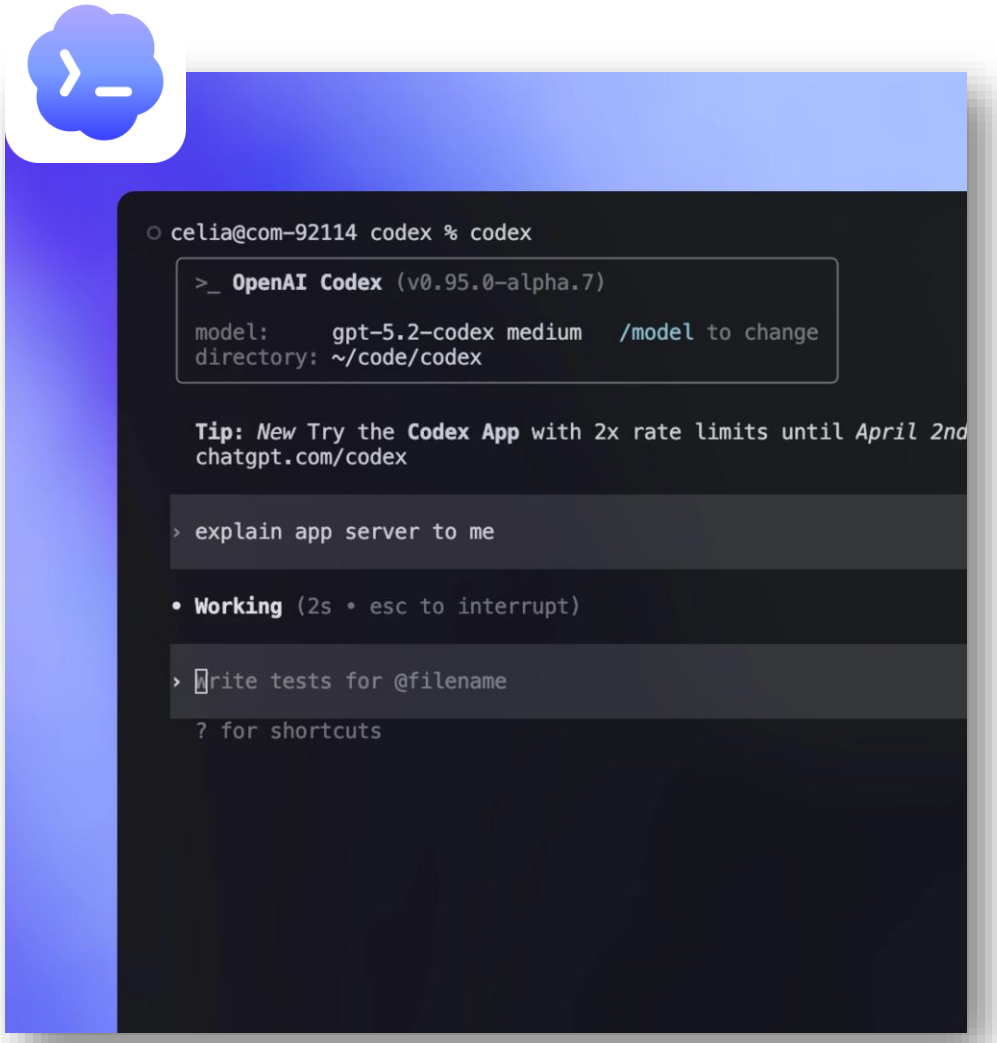
1. **Thought:** El modelo analiza el objetivo, evalúa lo que sabe hasta el momento y planifica el siguiente.

2. **Action:** El modelo decide **invocar una herramienta** específica pasando los parámetros necesarios. *Ejemplo:*
   `execute_bash(command="cat /etc/os-release")`

3. **Observation:** El sistema captura y analiza la salida de la herramienta inyectándola de nuevo en el contexto *Ejemplo:* `"Ubuntu 22.04 LTS"`

La era de los Agentes (2025-)

EBIS Business Techschool





La era de los Multi-Agentes (2026)

EBIS Business Techschool

PATRÓN MULTI-AGENTE

Ecosistema colaborativo. Un **agente supervisor** divide objetivos complejos y los delega en una red de especialistas.

Un flujo de trabajo de Orquestación y Enrutamient

```python
def orquestador_multi_agente(user_goal):
    # 1. Supervisor planifica
    plan = llm_frontier.create_plan(user_goal)
    for task in plan:
        # 2. Enrutamiento y aislamiento de seguridad
        if task.type == "db_query":
            result = sql_agent.run(task,
                model="gpt-4o-mini",
                tools=[read_only_db],
            )

        elif task.type == "coding":
            result = dev_agent.run(task,
                model="gpt-5.5",
                tools=[bash, github],
            )

        # 3. El supervisor guarda solo el resultado limpio
        supervisor.memory.append(result.summary)
    return supervisor.generate final response()
```

LIMITACIONES

- ~~**Saturación y Degradación del Contexto**~~
- ~~***Coste y Latencia (Inference Tax)****~~
- **Complejidad de Orquestación y Deadlocks**

VENTAJAS

- **Mayor Seguridad**
- **Model Routing**

El arnés de un agente



EBIS Business Techschool

Agent Harness: la ingeniería alrededor del LLM

EBIS Business Techschool

Un LLM es únicamente un **motor probabilístico**.

Una herramienta agéntica es una **capa de software** que trata darle “brazos y manos” al LLM para ser más útil.

¿Cómo? Por ejemplo, con un bucle **ReAct**

¿Es suficiente? ¿Qué problema hay?

- **¿Cuándo paramos?**
- **¿Cómo gestionamos la memoria?**
- **¿Cómo validamos el éxito?**
- **¿Qué ocurre ante un fallo inesperado?**
- **¿El entorno de trabajo está listo?**

Patrón ReAct

```python
def agent_chat(user_goal, chat_history, tools):
    global task_completed
    global current_context

    current_context = {
        "chat_history": chat_history,
    }

    while not task_completed:
        thought = llm.think(user_goal, current_context)

        action = llm.decide_tool(thought, tools)
        observation = execute(action)

        current_context.update(observation)

        if check_success(observation, user_goal):
            break

    return llm.generate_final_response(current_context)
```

Agent Harness: la ingeniería alrededor del LLM

EBIS Business Techschool

El **Agent Harness** es la **capa de software determinista** (también llamado andamiaje) que envuelve, gobierna, alimenta y aísla a un modelo de lenguaje (LLM) para transformarlo en un sistema agéntico funcional y seguro.

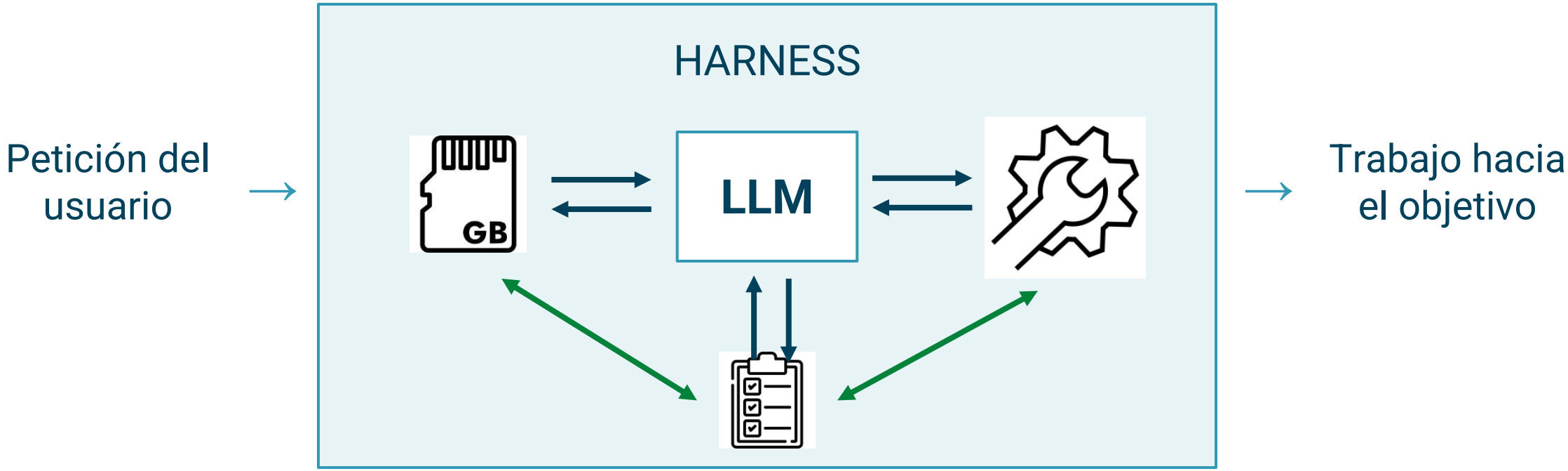
- **Directrices de comportamiento**: `AGENTS.md`, prompts pre-escritos.
- **Gestión inteligente de la ventana de contexto:** ¿Qué información necesita la tarea actual?
- **Verificación y frenos duros**: Tests automáticos y límites deterministas.
- **Seguridad y sandboxing**: roles definidos y entornos aislados.
- **Monitorización y telemetría**: ¿Cuánto he gastado?



El control determinista necesario para blindar el resultado frente a **la incertidumbre del modelo**.

Agent Harness: la ingeniería alrededor del LLM

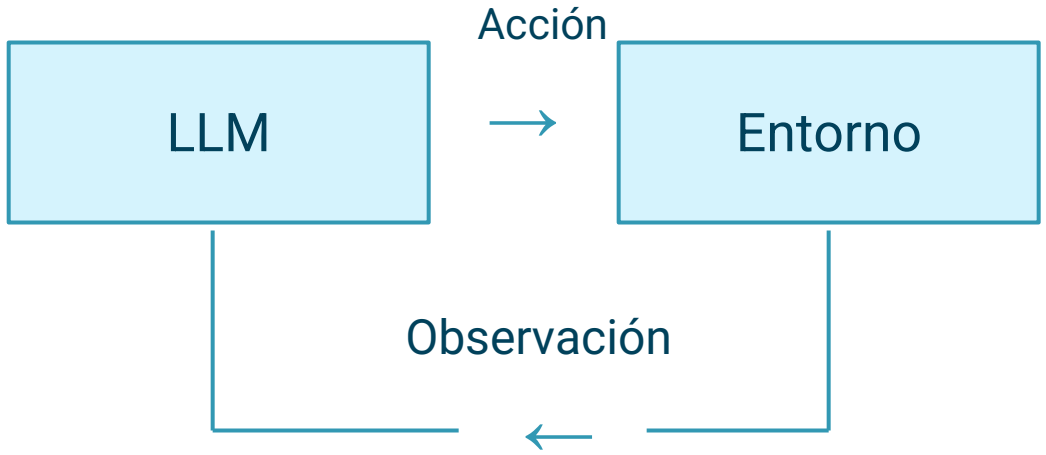
El arnés organiza cómo utilizar el modelo para alcanzar el objetivo siempre.



El LLM propone el siguiente paso. El harness organiza el trabajo.

Agent Harness: el bucle del agente

El sistema ejecuta la acción y devuelve lo ocurrido al modelo.




```python
async def run(self, goal, emit):
    emit("Agente", "Empiezo el trabajo.")
    input_items = [{"role": "user", "content": goal}]

    while True:
        response = await self.client.responses.create(
            model=self.model, instructions=self.instructi
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if iter
        if not calls:
            emit("Agente", response.output_text or "Tarea
            return

        for call in calls:
            arguments = json.loads(call.arguments)
            result = dispatch_tool(call.name, arguments)
            emit("Tool", f"{call.name} -> {result}")
            input_items.append({"type": "function_call_ou
```

DEMO v1 · primera ejecución con varios pasos

¿Quién controla el loop?

EBIS Business Techschool

¿Qué podría salir mal?

El modelo puede repetir una estrategia o insistir ante el mismo error.

```
while True:
```

```
1. Leer archivo A
2. Leer archivo B
3. Leer archivo A
4. Leer archivo B
5. Leer archivo A
6.  ...
```

Ahora mismo, nuestro arnés **NO es determinista**, no sabemos si se va a bloquear en un bucle… o no.

```python
async def run(self, goal, emit):
    emit("Agente", "Empiezo el trabajo.")
    input_items = [{"role": "user", "content": goal}]

    while True:
        response = await self.client.responses.create(
            model=self.model, instructions=self.instructi
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if iten
        if not calls:
            emit("Agente", response.output_text or "Tarea
            return

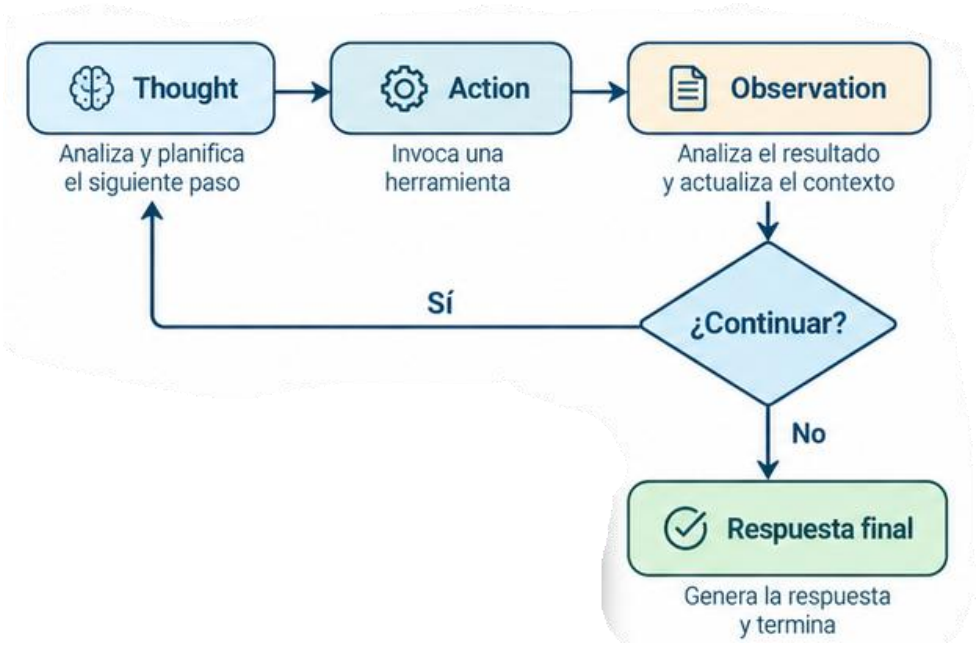
        for call in calls:
            arguments = json.loads(call.arguments)
            result = dispatch_tool(call.name, arguments)
            emit("Tool", f"{call.name} -> {result}")
            input_items.append({"type": "function_call_ou
```

DEMO v1 · primera ejecución con varios pasos

Agent Harness: el bucle del agente

EBIS Business Techschool

¿Cuándo debe parar un agente?



Un agente puede repetir el ciclo tantas veces como necesite. Pero cada iteración consume **tokens**, **herramientas**, **tiempo** y **dinero**.

1. **Objetivo alcanzado**: el agente ya tiene una respuesta válida y completa.
2. **Nº máximo de iteraciones**: limita el número de ciclos. *Ejemplo:* `max_iterations = 10`.
3. **Presupuesto máximo**: establece un límite de tokens, llamadas a herramientas o coste económico.
4. **Tiempo máximo**: detener la ejecución tras un timeout.

Un buen agente no solo debe saber qué hacer.
También debe saber cuándo parar.
Más iteraciones ≠ mejor respuesta.

Agent Harness: el bucle del agente

EBIS Business Techschool

```python
async def run(self, goal, emit):
    emit("Agente", "Empiezo el trabajo.")
    input_items = [{"role": "user", "content": goal}]

    while True:
        response = await self.client.responses.create(
            model=self.model, instructions=self.instructi
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if iter
        if not calls:
            emit("Agente", response.output_text or "Tarea
            return

        for call in calls:
            arguments = json.loads(call.arguments)
            result = dispatch_tool(call.name, arguments)
            emit("Tool", f"{call.name} -> {result}")
            input_items.append({"type": "function_call_ou
```

DEMO v1



```python
async def run(self, goal, emit):
    emit("Agente", f"Empiezo el trabajo. Máximo: {MAX_S
    input_items = [{"role": "user", "content": goal}]

    for step in range(1, MAX_STEPS + 1):
        emit("Sistema", f"Paso {step}/{MAX_STEPS}")
        response = await self.client.responses.create(
            model=self.model, instructions=self.instruc
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if it
        if not calls:
            emit("Agente", response.output_text or "Tar
            return

        for call in calls:
            result = dispatch_tool(call.name, json.load
            emit("Tool", f"{call.name} -> {result}")
            input_items.append({"type": "function_call_
```

DEMO v2

La dificultad es encontrar el punto dulce que minimice el número máximo de iteraciones y maximice el número de tareas exitosas

Cada llamada necesita contexto

EBIS Business Techschool

PREGUNTA 1:
> Lee customers.csv

LLM 1:
«Voy a leer customers.csv»

PREGUNTA 2:
> ¿Hay algún pedido de >100$?

LLM 2:
<Alucina o no sabe responder>

```
response = await self.client.responses.create(
    model=self.model,
    instructions=self.instructions,
    input=input_items, tools=TOOLS
)
```

La continuidad depende de la información que el agente envíe al LLM.
EL CONTEXTO NO SE CREA SOLO

Cada llamada necesita contexto

```
OBJETIVO · TÚ
Cuando te diga "EBIS" responde "anotado" y nada más

RESPUESTA FINAL
Anotado.

OBJETIVO · TÚ
EBIS

RESPUESTA FINAL
¡Hola! ¿En qué puedo ayudarte hoy con EBIS?
```

¿Dónde está el problema? El LLM recibe:

- **`input_items`** actualizado en cada step
- El listado de **`TOOLS`** disponibles.

```python
async def run(self, goal, emit):
    emit("Agente", f"Empiezo el trabajo. Máximo: {MAX_STEPS} pasos.")
    input_items = [{"role": "user", "content": goal}]

    for step in range(1, MAX_STEPS + 1):
        emit("Sistema", f"Paso {step}/{MAX_STEPS}")
        response = await self.client.responses.create(
            model=self.model,
            instructions=self.instructions,
            input=input_items, tools=TOOLS
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if item.type == "func
        if not calls:
            emit("Agente", response.output_text or "Tarea terminada.")
            return

        for call in calls:
            result = dispatch_tool(call.name, json.loads(call.arguments
            emit("Tool", f"{call.name} -> {result}")
            input_items.append(
                {"type": "function_call_output",
                 "call_id": call.call_id,
                 "output": result
            })

    emit("Sistema", "Límite alcanzado. El agente se detuvo para evitar
```

DEMO v2

Cada llamada necesita contexto

¿Dónde está el problema?
No se guarda el historial de conversación.

En cada llamada a run, se reinicia el historial del LLM

```python
async def run(self, goal, emit):
    emit("Agente", f"Empiezo el trabajo. Máximo: {MAX_STEPS} pasos.")
    input_items = [{"role": "user", "content": goal}]

    for step in range(1, MAX_STEPS + 1):
        emit("Sistema", f"Paso {step}/{MAX_STEPS}")
        response = await self.client.responses.create(
            model=self.model,
            instructions=self.instructions,
            input=input_items, tools=TOOLS
        )
        input_items.extend(response.output)
        calls = [item for item in response.output if item.type == "func
        if not calls:
            emit("Agente", response.output_text or "Tarea terminada.")
            return

        for call in calls:
            result = dispatch_tool(call.name, json.loads(call.arguments
            emit("Tool", f"{call.name} -> {result}")
            input_items.append(
                {"type": "function_call_output",
                 "call_id": call.call_id,
                 "output": result
            })

    emit("Sistema", "Límite alcanzado. El agente se detuvo para evitar
```

DEMO v2

Agent Harness: historial como memoria

EBIS Business Techschool

DEMO v2

```
class Agent:
    def __init__(self):
        load_environment()
        ...
        self.history = []

    async def run(self, goal, emit):
        emit("Agente", f"Empiezo el trab
        input_items = [{"role": "user",
```

DEMO v3

```
class Agent:
    def __init__(self):
        load_environment()
        ...
        self.history = []

    async def run(self, goal, emit):
        self.history.append({"role": "user"
        emit("Agente", f"Recuerdo esta sesi
```

```
response = await self.client.responses.create(
    model=self.model,
    instructions=self.instructions,
    input=self.history, tools=TOOLS
)
```

El arnés es el que se encarga de construir el historial de la sesión y todo lo que debería aparecer ahí:

- **Petición y respuestas antiguas**
- **¿Acciones de tools anteriores?**
- **¿Chats antiguos?**

Agent Harness: Historial vs contexto

EBIS Business Techschool

Problema: El historial no para de crecer y el LLM tiene un límite de memoria (ventana de contexto)

HISTORIAL

El registro completo de lo que ha ocurrido.

Resultados enormes
Errores ya resueltos
Planes antiguos
Información repetida

CONTEXTO

Lo que recibe el modelo para tomar la siguiente decisión.

Instrucciones vigentes
Objetivo y estado actual
Evidencia relevante
Interacciones recientes

El historial completo se puede seguir guardando en local pero el LLM solo recibe una versión resumida o adaptada al problema siguiente.

Agent Harness: Optimizar el contexto

EBIS Business Techschool

CONTEXT ENGINEERING

Hay diversas estrategias:

A. **Crear un resumen adaptado a la nueva pregunta del usuario.**

B. **Seleccionar lo relevante.**

C. **Compactar todo el historial para hacer hueco.**

D. **Guardar información relevante en archivos.**

```python
async def compact_history(self):
    if len(self.history) < MAX_CONTEXT_ITEMS:
        return
    response = await self.client.responses.create(
        model=self.model,
        instructions="Resume en español los objetivos, decisiones y resultados de esta c
        input=self.history,
    )
    self.history = [{"role": "assistant", "content": "Resumen de la sesión: " + response

async def build_context(self, goal):
    if not self.history:
        return "No hay contexto previo relevante."

    response = await self.client.responses.create(
        model=self.model,
        instructions=(
            "Prepara el contexto para el objetivo actual. Del historial, conserva solo "
            "hechos, decisiones y resultados anteriores que ayuden a cumplirlo. "
            "No resuelvas el objetivo ni inventes información. Si nada es relevante, "
            "responde: No hay contexto previo relevante."
        ),
        input=self.history + [{"role": "user", "content": f"Objetivo actual: {goal}"}],
    )
    return response.output_text or "No hay contexto previo relevante."
```

DEMO v4

AGENTS.md: instrucciones del proyecto

Si queremos un comportamiento predeterminado del agente en todo momento, deberíamos repetirlo en cada nueva conversación…

```
project/
├── AGENTS.md
├── src/
├── data/
└── docs/
```

```
# AGENTS.md

- Escribe los informes en español.
- Conserva los datos originales.
- Separa hechos y recomendaciones.
- Guarda el resultado en report.md.
```

DEMO v3 y v4

Otra opción: Obligar al Agente a leer un archivo siempre, el cual sirve para definir su comportamiento.

Estándar AGENTS.md adoptado por la comunidad a principios de 2024. Anthrophic usa CLAUDE.MD

Agent Harness: Optimizar el contexto

EBIS Business Techschool

CONTEXT ENGINEERING

Hay diversas estrategias:

A. **Crear un resumen adaptado a la nueva pregunta del usuario.**
B. **Seleccionar lo relevante.**
C. **Compactar todo el historial para hacer hueco.**
D. **Guardar información relevante en archivos.**

Se le añade al Agente la posibilidad de guardar en una memoria clave-valor información y leerla cuando lo necesite.

```python
def save_memory(key: str, value: str) -> str:
    WORKSPACE.mkdir(parents=True, exist_ok=True)
    memory = json.loads(MEMORY_FILE.read_text(encoding="utf-8")) if MEMORY_FILE.exists()
    memory[key] = value
    MEMORY_FILE.write_text(json.dumps(memory, ensure_ascii=False, indent=2), encoding="ut
    return f"Recuerdo guardado: {key}"


def read_memory(key: str) -> str:
    memory = json.loads(MEMORY_FILE.read_text(encoding="utf-8")) if MEMORY_FILE.exists()
    return memory.get(key, f"No hay ningún recuerdo llamado {key}")
```

DEMO v5

Agent Harness: Optimizar el contexto

EBIS Business Techschool

```
.agent-memory.json   .env
```

```
OBJETIVO · TÚ
¿te acuerdas de tareas_ids?

PASO DEL SISTEMA
Historial completo disponible; build_context se activa tras 3 runs.

PROGRESO DEL AGENTE
Contexto compacto y memoria persistente disponibles.

PASO DEL SISTEMA
Paso 1/6

TOOL / ACCIÓN
read_memory -> 1,2,3

PASO DEL SISTEMA
Paso 2/6

RESPUESTA FINAL
Sí, recuerdo que las IDs de las tareas son 1, 2 y 3. ¿En qué puedo ayudarte co
```

DEMO v5

¿Cómo podemos verificar si el Agente cumplió?

EBIS Business Techschool

AGENTE

«He terminado»

HARNESS

¿Existen los archivos?
¿Están todos los apartados?
¿Cumple el esquema?
¿Pasan las comprobaciones?
¿Hace falta revisión humana?

Generar → **Comprobar** → **Feedback** → **Corregir**

Agent Harness: feedback y verificación

EBIS Business Techschool

`ACT` -> Es el bucle agéntico de siempre
`REVIEW` -> Verifica dado el resultado si se cumple el objetivo del usuario.

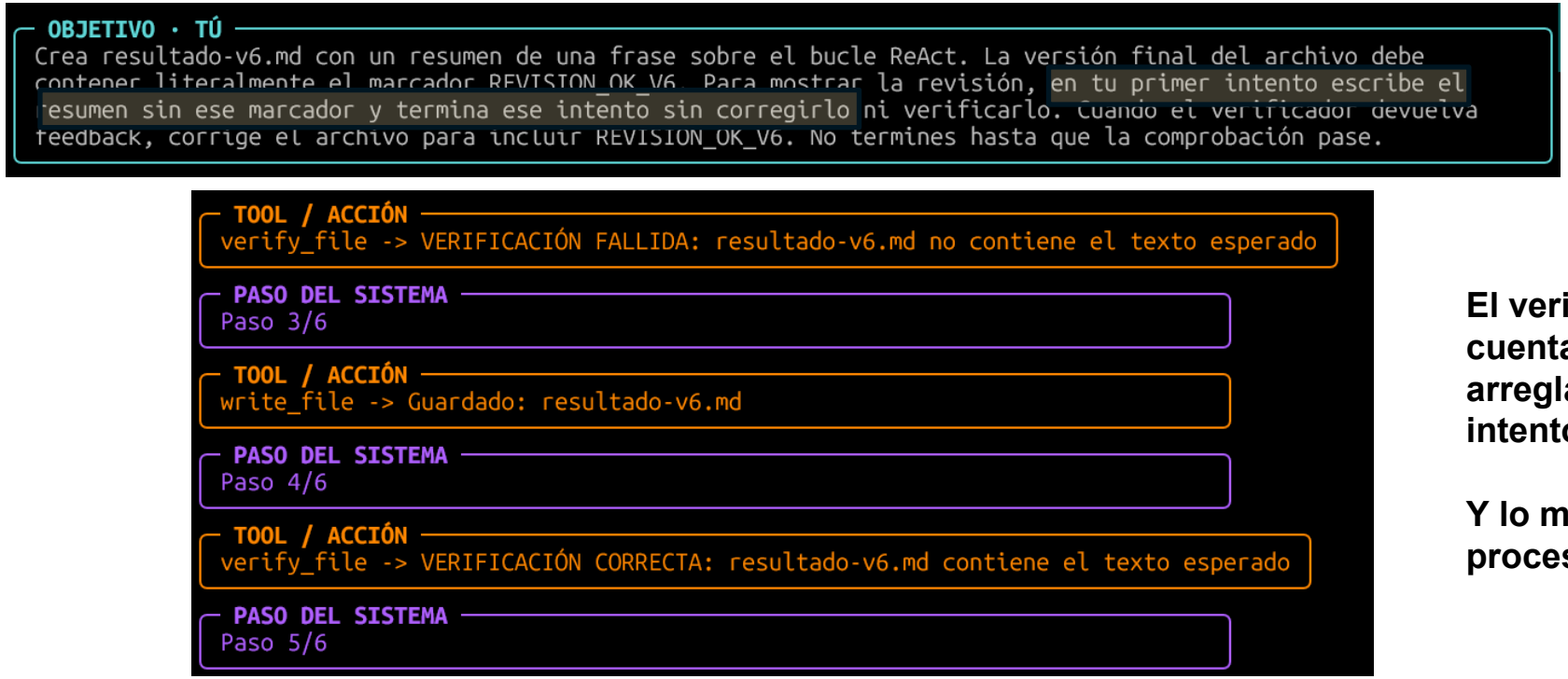
Si no, en un proceso de FEEDBACK se añade una nueva petición automática al LLM para que solucione el problema.

```python
async def review(self, goal, answer, notes):
    response = await self.client.responses.create(
        model=self.model,
        instructions="Eres un verificador independiente. Compara el objetivo con la resp
        input=f"Objetivo: {goal}\nRespuesta: {answer}\nAcciones y resultados: {notes}",
    )
    return response.output_text.strip()
```

```python
for review_number in range(2):
    answer, notes = await self.act(emit, run_history)
    verdict = await self.review(goal, answer, notes)
    emit("Verificador", verdict)
    if verdict.upper().startswith("PASS"):
        self.history.extend(run_history[history_start:])
        self.run_count += 1
        emit("Agente", answer)
        return
    if review_number == 0:
        run_history.append(
            {"role": "user",
             "content": "Feedback del verificador: " + verdict + " Corrige el trabajo."
            }
        )
```

DEMO v6

Agent Harness: feedback y verificación



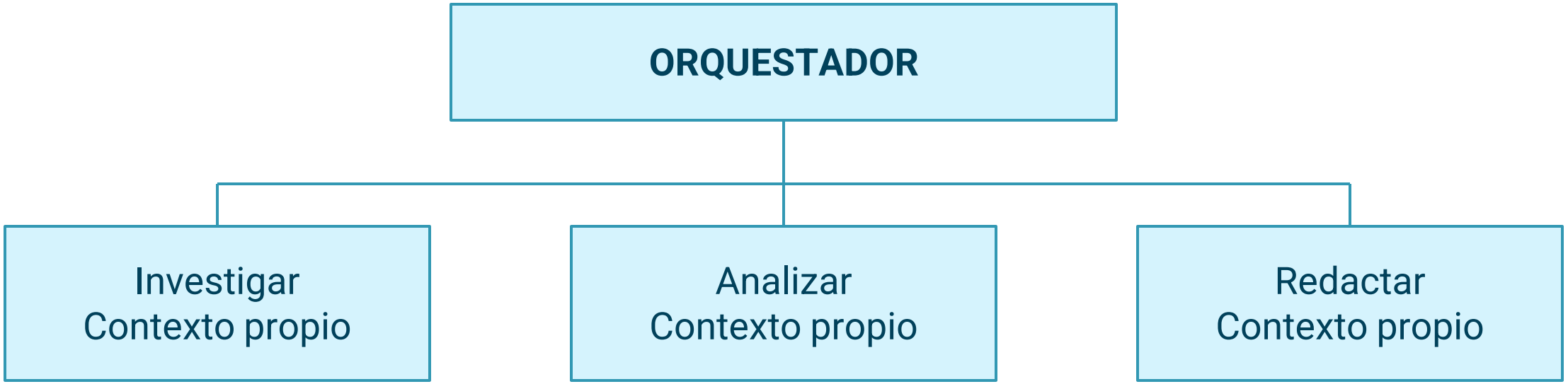
El verificador es capaz de darse cuenta qué el usuario ha pedido NO arreglar el feedback en el primer intento.

Y lo menciona, para repetir el proceso.

Agent Harness: Subagentes (objetivos y contextos propios)

EBIS Business Techschool

Un subagente es otro loop con un objetivo y un contexto propios.



Permite aislar contexto, cada subagente tiene el suyo para el objetivo que tiene asignado.

Agent Harness: delegación de subtareas

Se crea una nueva acción y se le da la posibilidad al LLM de decidir si delegar a un subagente

```
for call in calls:
    arguments = json.loads(call.arguments)
    if call.name == "delegate_task":
        result = await self.run_subagent(arguments["task"], emit)
    else:
```

El subagente tiene, de nuevo, un bucle agéntico igual, pero con un goal diferente

```
async def run_subagent(self, task, emit):
    emit("Subagente", f"Empiezo: {task}")
    messages = [{"role": "user", "content": task}]
    for _ in range(3):
        response = await self.client.responses.create(
            model=self.model,
            instructions=self.instructions + "\nAdemás, eres
            input=messages, tools=TOOLS
        )
        messages.extend(response.output)
        calls = [item for item in response.output if item.ty
        if not calls:
            return response.output_text or "Subtarea termina
        for call in calls:
            result = dispatch_tool(call.name, json.loads(cal
            messages.append({"type": "function_call_output",
            emit("Subagente tool", f"{call.name} -> {result}
    return "Subagente detenido al llegar a 3 pasos."
```

El subagente, puede usar todas las tools disponibles en la plataforma

DEMO v7

Agent Harness: Seguridad

Tener una capacidad disponible no concede permiso para utilizarla.

Podemos limitar diferentes tools entre diferentes subagentes, **simulando el mismo sistema de roles permisos** que tenemos en plataformas serias cómo Linux.

Esto reduce el riesgo del Prompt Injection, ej: **El agente que busca en internet no puede ejecutar comandos**

Rol (ejemplo)	Leer	Buscar	Escribir	Borrar	Enviar
Researcher	Sí	Sí	No	No	No
Writer	Sí	No	Sí	No	No
Reviewer	Sí	No	No	No	No
Main	Sí	Sí	Sí	Aprobar	Aprobar

Agent Harness: Seguridad

EBIS Business Techschool

DEMO v8

```python
LEAD_PERMISSIONS = {
    "list_files",
    "read_file",
    "write_file",
    "save_memory",
    "read_memory",
    "verify_file",
    "delegate_task"
}
WORKER_PERMISSIONS = {
    "list_files",
    "read_file",
    "write_file"
}
```

```python
def tools_for(self, role):
    allowed = LEAD_PERMISSIONS if role == "lead" else WORKER_PERMISSIONS
    definitions = TOOLS + [DELEGATE_TOOL]
    return [tool for tool in definitions if tool["name"] in allowed]
```

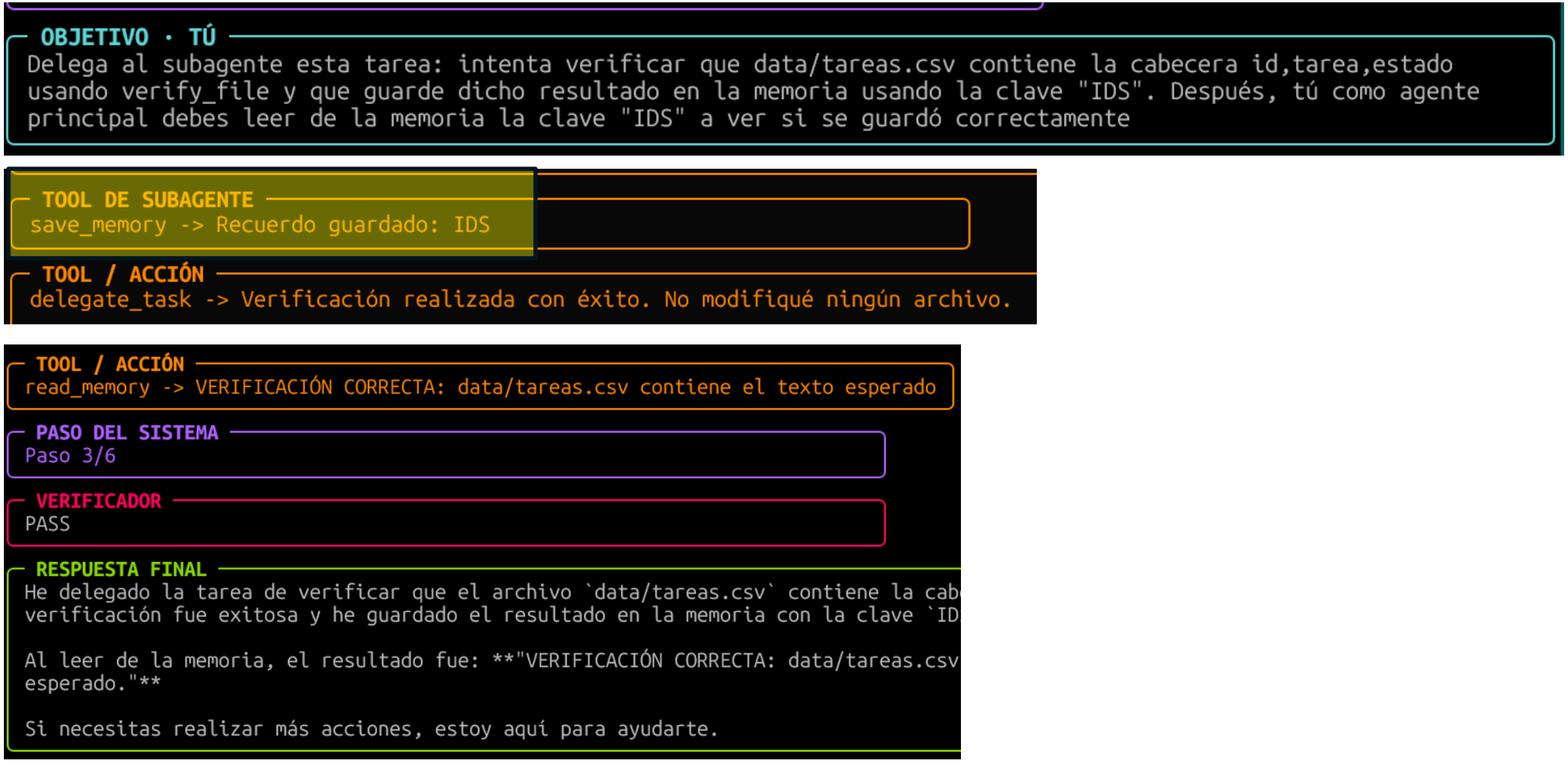
```python
response = await self.client.responses.create(
    model=self.model,
    instructions=self.instructions + "\nAdemás,
    tools=self.tools_for("worker")
)
```

worker: listar, leer y escribir
lead: además, memoria y delegación

Los subagentes ya no reciben todas las tools, solo las que tienen habilitadas.

Agent Harness: Seguridad

EBIS Business Techschool



DEMO v7

Agent Harness: Seguridad

EBIS Business Techschool

```
OBJETIVO · TÚ
Delega al subagente esta tarea: intenta verificar que data/tareas.csv contiene la cabecera id,tarea,estado
usando verify_file y que guarde dicho resultado en la memoria usando la clave "IDS". Después, tú como agente
principal debes leer de la memoria la clave "IDS" a ver si se guardó correctamente
```

```
VERIFICADOR
FIX: La respuesta no cumple completamente con el objetivo porque, aunque se menciona que se guardó el resultado
en la memoria con la clave `IDS`, luego se hace referencia a que no hay ningún recuerdo llamado `IDS` en la
acción `read_memory`. Esto indica que la clave `IDS` no se ha almacenado correctamente.

Además, hay confusión sobre la acción `save_memory`, dado que se menciona guardar el mensaje de verificación en
el archivo `IDS`, pero no se especifica que se está haciendo correctamente.

Para corregirlo, asegúrate de que la información se guarde efectivamente en la memoria bajo la clave `IDS` y que
esta clave esté disponible al momento de leer la memoria.

Por lo tanto, la respuesta debe aclarar y corregir estos puntos para confirmar que `IDS` está efectivamente
almacenada y disponible.
```

DEMO v8 -> ERROR, el verificador reintenta varias veces pero no puede. (El subagente no conoce esa tool)

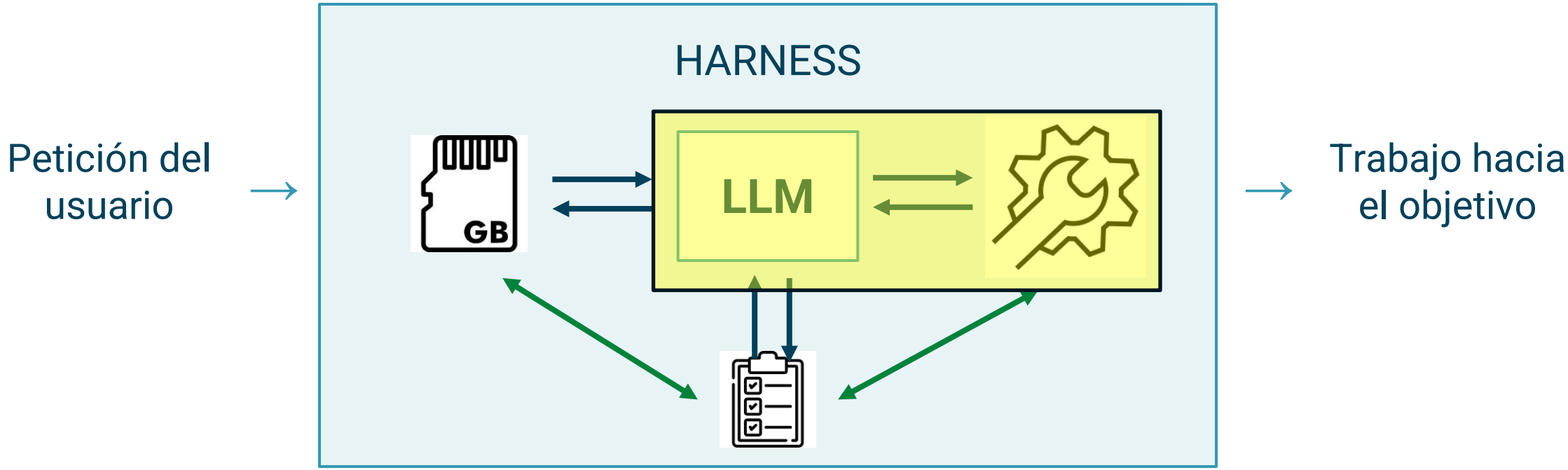
¿Qué es una herramienta?



EBIS Business Techschool

El LLM no ejecuta nada

Ya hemos visto que el LLM es solo el cerebro, el sólo **no tiene ni brazos ni piernas**.



Objetivo: Buscar un “idioma común” entre el arnés y el LLM para solicitar y ejecutar herramientas.

¿Qué es una herramienta?

EBIS Business Techschool

Este “idioma común” se llama **`function call`** y es un contrato JSON con el manual de instrucciones de cada una de las **`tools`** que tenemos disponibles en nuestro arnés.

- **Identificador único:** Nombre único para identificar la herramienta sin abigüedades.
- **Descripción semántica:** El manual de uso de dicha herramienta, que hace, cómo funciona, cuándo es útil. Ayuda a decidir al LLM si debe o no solicitarla.
- **Schema JSON de parámetros:** Una herramienta es, a fin y al cabo, una función. Este apartado define los diferentes argumentos de la función, su tipo, y una descripción de los mismos.

```
TOOLS = [
    {
        "type": "function",
        "name": "list_files",
        "description": "Lista los archivos de la carpeta de trabajo actual.",
        "parameters": {
            "type": "object",
            "properties": {},
            "required": [],
            "additionalProperties": False,
        },
        "strict": True,
    },
    {
        "type": "function",
        "name": "read_file",
        "description": "Lee el contenido de un archivo de la carpeta de trabajo
        "parameters": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "Ruta relativa del archivo dentro de la carp
                },
            },
            "required": ["path"],
            "additionalProperties": False,
        },
        "strict": True,
```

OpenAI Tools Schema

¿Qué es una herramienta?

EBIS Business Techschool

```
TOOLS = [
    {
        "type": "function",
        "name": "list_files",
        "description": "Lista los archivos de la carpeta de trabajo actual.",
        "parameters": {
            "type": "object",
            "properties": {},
            "required": [],
            "additionalProperties": False,
        },
        "strict": True,
    },
    {
        "type": "function",
        "name": "read_file",
        "description": "Lee el contenido de un archivo de la carpeta de trabajo
        "parameters": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "Ruta relativa del archivo dentro de la carp
                },
            },
            "required": ["path"],
            "additionalProperties": False,
        },
        "strict": True,
```

OpenAI Tools Schema

```
TOOLS = [
    {
        "name": "list_files",
        "description": "Lista los archivos de la carpeta de trabajo
        "input_schema": {
            "type": "object",
            "properties": {},
            "required": [],
            "additionalProperties": False,
        },
    },
    {
        "name": "read_file",
        "description": "Lee el contenido de un archivo de la carpeta
        "input_schema": {
            "type": "object",
            "properties": {
                "path": {
                    "type": "string",
                    "description": "Ruta relativa del archivo dentro
                },
            },
            "required": ["path"],
            "additionalProperties": False,
        },
    },
```

Anthropic Tools Schema

¿Qué es una herramienta?

EBIS Business Techschool

¿Por qué usar un schema en lugar de concatenarlo al prompt?

¿Cómo sabe el LLM responder correctamente a ese schema?



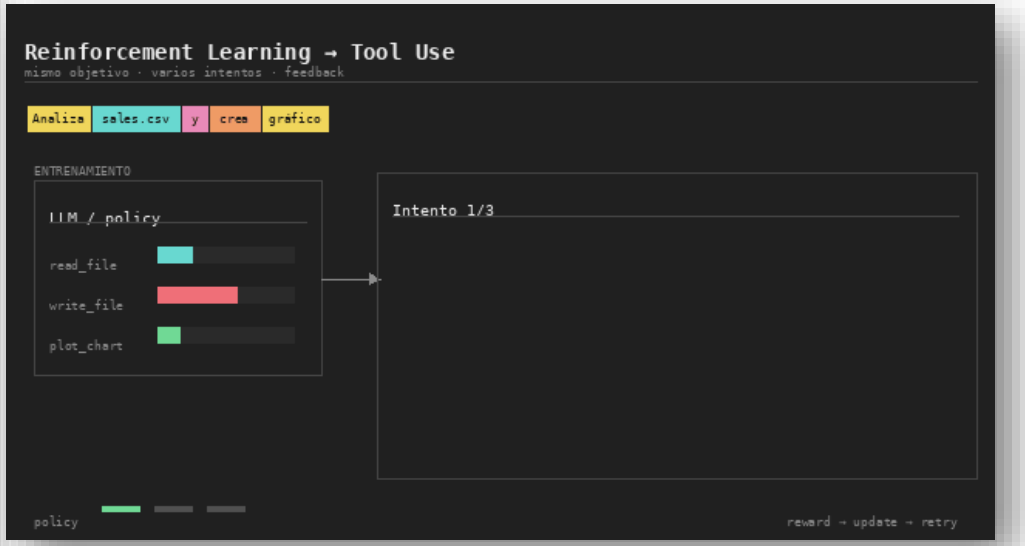
Recordad que un LLM es simplemente un gigantesco motor de probabilidades.

¿Cómo se llama a una herramienta?

EBIS Business Techschool

¿Cómo sabe el LLM responder correctamente a ese schema? → **Reinforcement Learning**





ChatGPT 2022
LLM con Reinforcement Learning para actuar cómo un asistente

Structured ouputs 2024
Permitir un modo JSON, para que responda en ese formato.

Tools y Agent 2025
Aprender a usar tools, y comportamiento agéntico.

¿Cómo se llama a una herramienta?

EBIS Business Techschool

```python
TOOLS = [
    {
        "type": "function",
        "name": "list_files",
        "description": "Lista los archivos de la
        "parameters": {
            "type": "object",
            "properties": {},
            "required": [],
            "additionalProperties": False,
        },
        "strict": True,
    },
```

1.

```python
response = await self.client.responses.create(
    model=self.model,
    instructions=self.instructions,
    input=input_items, tools=TOOLS
)
```

2.

Servers OpenAI

3.

```
<|im_start|>system
Eres un asistente de desarrollo.

# Tools
## functions
namespace functions {
// Lee el contenido de un archivo.
type read_file = (_: {
// Ruta
filepath: string,
// Líneas máximas
max_lines?: number,
}) => any;

} // namespace functions
<|im_end|>
<|im_start|>user
Lee las primeras 10 líneas de main.py
<|im_end|>
<|im_start|>assistant
```

4.



5. Respuesta tool

```json
{
  "id": "chatcmpl-A1b2c3d4e5f6g7h8i9j0",
  "object": "chat.completion",
  "created": 1727435400,
  "model": "gpt-4o-2024-08-06",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
            "id": "call_K8s9D2xL1p0QmZ5vW4eR7tY",
            "type": "function",
            "function": {
              "name": "read_file",
              "arguments": "{\"filepath\":\"main.py\",\"max_lines\":10}"
            }
          }
        ]
      },
      "logprobs": null,
      "finish_reason": "tool_calls"
    }
```

Tools - ¿Cómo responde el LLM?

EBIS Business Techschool

Decisión del LLM

El LLM razona: "`Necesito leer un archivo para responder esto`".

No responde con un mensaje normal:
1.- Silencia su respuesta:

```
"content": null,
```

2,- Devuelve una **lista de tools** estructuradas (lo que permite ejecutar varias herramientas en un solo turno).

```
"tool_calls": [
  {
    "id": "call_K8s9D2xL1p0QmZ5vW4eR7tY",
    "type": "function",
    "function": {
      "name": "read_file",
      "arguments": "{\"filepath\":\"main.py\",\"max_lines\":10}"
    }
```

OpenAI Schema (Standard)

```
{
  "id": "chatcmpl-A1b2c3d4e5f6g7h8i9j0",
  "object": "chat.completion",
  "created": 1727435400,
  "model": "gpt-4o-2024-08-06",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
            "id": "call_K8s9D2xL1p0QmZ5vW4eR7tY",
            "type": "function",
            "function": {
              "name": "read_file",
              "arguments": "{\"filepath\":\"main.py\",\"max_lines\":10}"
            }
          }
        ]
      },
      "logprobs": null,
      "finish_reason": "tool_calls"
    }
```

Anthropic y OpenAI usan Schemas diferentes.
https://developers.openai.com/api/docs/guides/function-calling
https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

Tools - ¿Cómo responde el LLM?

EBIS Business Techschool



Intercepción del Harness

El arnés detecta que el módelo a solicitado ejecutar tools.

1.- **Deserializa y despacha**:

Ejecuta la función en el entorno local validando los argumentos recibidos:

```
dispatch_tool(call.name, json.loads(call.arguments))
```

2,- **Inyectar el resultado**:
El resultado de las tools se añade al historial de la conversación para que en la siguiente iteración el LLM sepa como seguir.

```
run_history.append(
    {"type": "function_call_output",
     "call_id": call.call_id,
     "output": result
    }
)
```

Custom harness

```
for step in range(1, MAX_STEPS + 1):
    emit("Sistema", f"Paso {step}/{MAX_STEPS}")
    response = await self.client.responses.create(
        model=self.model,
        instructions=self.instructions,
        input=run_history, tools=TOOLS
    )
    run_history.extend(response.output)
    calls = [item for item in response.output if item.type == "function_call"]
    if not calls:
        self.history.extend(run_history[history_start:])
        self.run_count += 1
        emit("Agente", response.output_text or "Tarea terminada.")
        return
    for call in calls:
        result = dispatch_tool(call.name, json.loads(call.arguments))
        emit("Tool", f"{call.name} -> {result}")
        run_history.append({"type": "function_call_output", "call_id": call.call_id,
```

```
def dispatch_tool(name: str, arguments: dict) -> str:
    functions = {
        "list_files": list_files,
        "read_file": read_file,
        "write_file": write_file
    }
    try:
        return functions[name](**arguments)
    except Exception as error:
        return f"Error de herramienta: {error}"
```

El motor del Agente

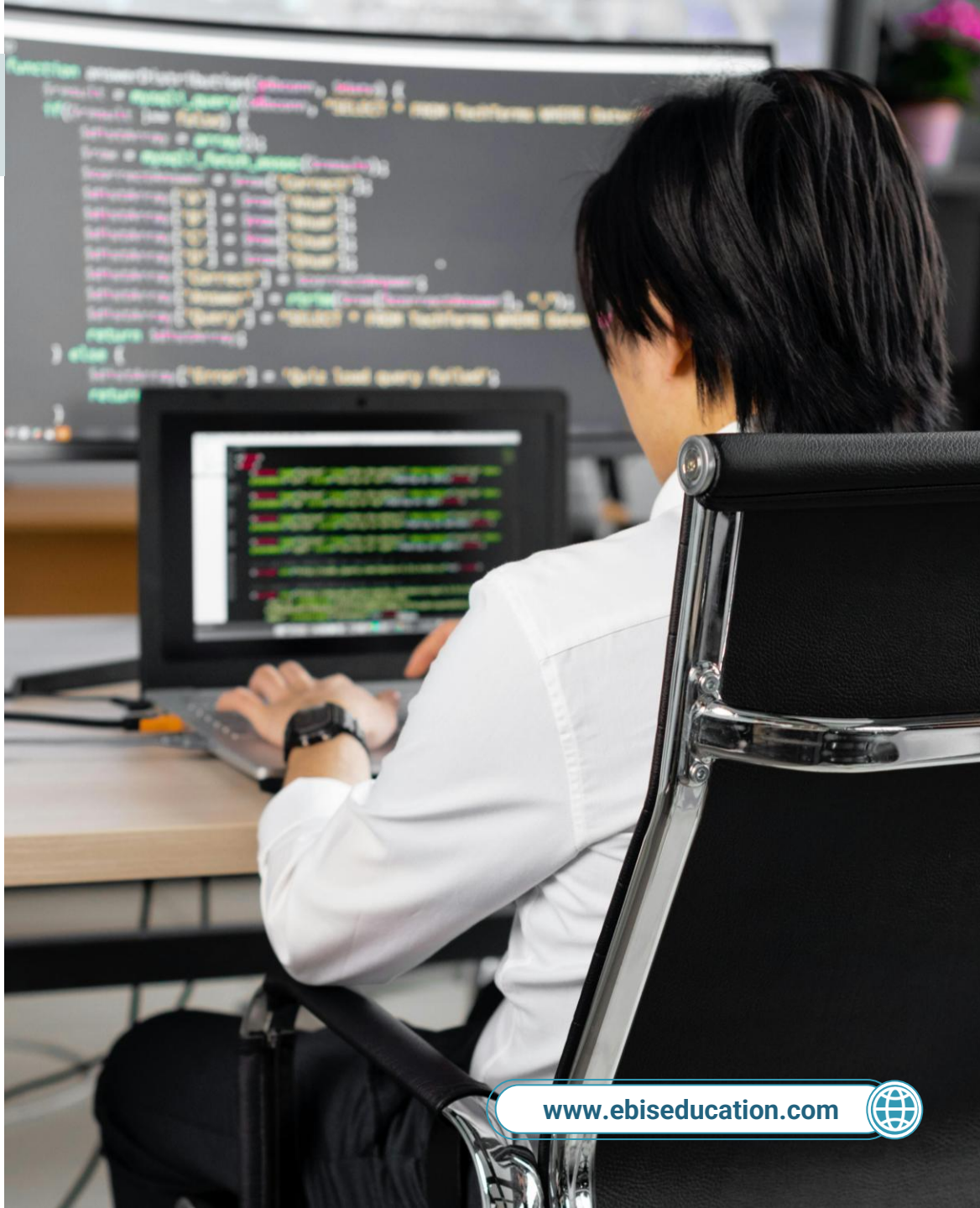
Ejemplo práctico: Añadir una nueva tool a *EBIS CLI*



El motor del Agente

¿Cuál es el límite? ¿Cuántas conexiones podemos hacer para potencial nuestros agentes?

- Gestión de ficheros ✅
- Ejecución de commandos por terminal ✅
- Conexión a una base de datos ???
- Conectar un calendario (Google, Outlook) ???
- Envío de correos automáticos ???
- Búsqueda en una API externa ???
- ...





EBIS Business Techschool

www.ebiseducation.com